42057 Introduction to Space Communications and Sensing

Spring 2026

Binary Data and Spread-Spectrum Communications

Assignment 2 — Topic 2

Ingle & Proakis Projects 12.7 and 12.8

Student:	[Student Name] ([Student ID])
Group:	The Goats
Members:	[Member 1], [Member 2], [Member 3], [Member 4]
Date:	October 13, 2026

Abstract

A baseband binary communication system and its direct-sequence spread-spectrum (DSSS) extension were designed, implemented in MATLAB and evaluated by Monte-Carlo simulation. Each block of the textbook system models (bit source, rectangular-pulse modulator, uniform noise channel, integrate-and-dump demodulator and threshold detector) is a separate MATLAB function, and the same functions are reused for the spread-spectrum system by adding a ± 1 pseudo-noise (PN) sequence and a sinusoidal jammer. For the unjammed system the measured bit error rate (BER) agrees with an exact analytical result to within Monte-Carlo accuracy from 2×10^{-1} down to 5×10^{-8}, and it shows that the common Gaussian approximation $\mathcal{Q}(\sqrt{\mathrm{SNR}})$ is pessimistic at high SNR because uniform noise is bounded. For the jammed system, despreading converts the narrowband jammer into wideband noise and provides a processing gain of $10 \log_{10} M$ dB, which raised the jamming power the receiver tolerates at BER $= 10^{-3}$ by 11 dB to 20 dB for $M = 50$ to 1000. Under the brief's literal condition $A = M$, however, the jammer power grows as M^2 while the processing gain grows only as M, so both systems operate near BER $= 0.5$; the PN system is better against an in-band jammer but can be worse against an out-of-band one. All results are matched by closed-form expressions derived in the report.

Contents

1 Introduction **2**

2 Theory **2**
2.1 Binary system with additive noise 2
2.2 Sinusoidal jamming without spreading 3
2.3 Direct-sequence spreading 4

3 MATLAB implementation **4**
3.1 Structure 4
3.2 Simulation design 5

4 Results: Project 12.7 **5**

5 Results: Project 12.8 **6**
5.1 The brief's experiment: $A = M$ 6
5.2 Effect of jammer power 7
5.3 Effect of jammer frequency 8
5.4 Spectral view 8

6 Discussion **9**
6.1 Effect of the PN sequence on the sinusoidal interference 9
6.2 Why the spread-spectrum system outperforms the conventional one 9
6.3 Interpreting the condition $A \geq M$ 10
6.4 Limitations 10

7 Conclusion **11**

Individual contribution **11**

A MATLAB code **12**

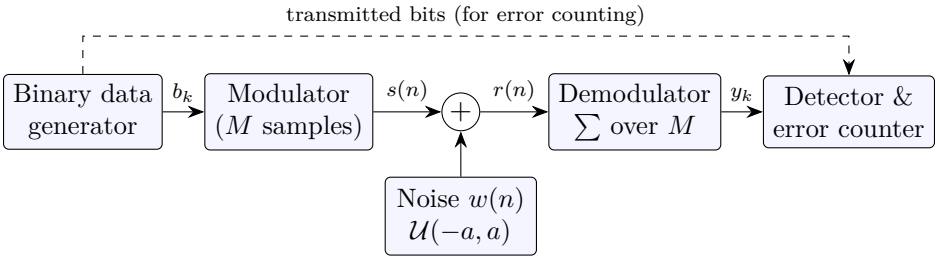


Figure 1: Binary data communication system of Project 12.7 (after Fig. 12.16 of [1]).

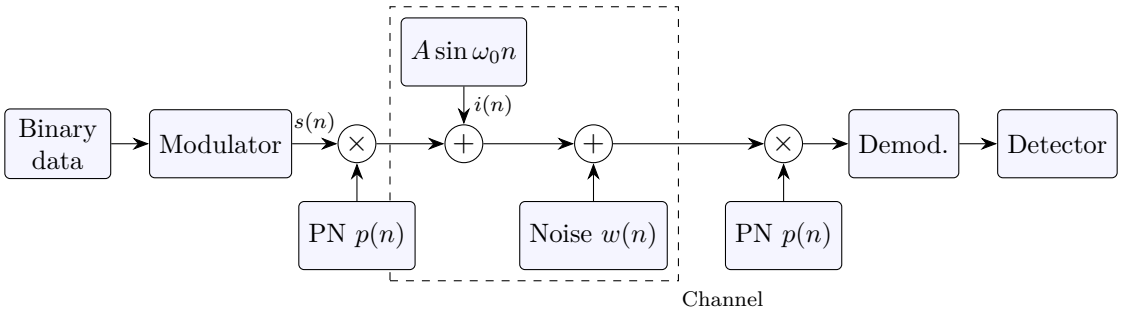


Figure 2: Binary PN spread-spectrum system of Project 12.8 (after Fig. 12.18 of [1]). The same PN sequence is used at the transmitter and the (synchronised) receiver.

1 Introduction

Spacecraft links are power limited, operate over long distances, and are exposed to both unintentional radio-frequency interference and deliberate jamming. Spread spectrum is the standard answer to the second problem: the Global Positioning System spreads its 50 bit/s navigation message with a 1.023 Mchip/s C/A code, a processing gain of about 43 dB, and NASA's Tracking and Data Relay Satellite System uses direct-sequence spreading on its multiple-access links for the same reasons: interference rejection, low power spectral density and code-division multiple access.

This assignment studies the mechanism behind that protection using two projects from Ingle and Proakis [1]:

- **Project 12.7** — a binary data communication system over an additive uniform-noise channel (Figure 1); the BER is measured as a function of the signal-to-noise ratio $\mathrm{SNR} = M/P_n$.
- **Project 12.8** — the same system with the modulator output multiplied by a binary PN sequence and a channel that also adds a sinusoidal jammer $i(n) = A \sin \omega_0 n$ with $A \geq M$ (Figure 2); the BER is measured with and without the PN sequence for $M = 50, 100, 500, 1000$.

The objectives were to (i) implement every block of both systems as a reusable MATLAB function, (ii) verify the simulator against theory, (iii) quantify the effect of the PN sequence on the jammer, and (iv) explain why, and when, the spread-spectrum system outperforms the conventional one. Section 2 develops the theory, Section 3 describes the implementation, Sections 4 and 5 present the results, and Section 6 discusses them.

2 Theory

2.1 Binary system with additive noise

Bit $b_k \in \{0, 1\}$ is mapped to the antipodal level $d_k = 2b_k - 1 = \pm 1$ and repeated for M samples,

$$s(n) = d_k, \qquad kM \leq n < (k+1)M. \tag{1}$$

The channel adds i.i.d. noise $w(n) \sim \mathcal{U}(-a, a)$ with power $P_n = \mathrm{E}[w^2] = a^2/3$. The demodulator sums the M received samples of each bit,

$$y_k = \sum_{n=kM}^{(k+1)M-1} r(n) = M d_k + \underbrace{\sum_{n} w(n)}_{W_k}, \tag{2}$$

which is the matched filter (integrate-and-dump) for a rectangular pulse. The detector decides $\hat{b}_k = 1$ if $y_k > 0$. The signal component has energy M^2 and the noise component W_k has variance MP_n, so the post-detection SNR is $M^2/(MP_n) = M/P_n$, which is the definition used in the brief:

$$\mathrm{SNR} = \frac{M}{P_n}. \tag{3}$$

By symmetry the bit error probability is $P_e = \Pr\{W_k > M\}$.

Gaussian approximation. For large M the central limit theorem gives $W_k \approx \mathcal{N}(0, MP_n)$ and

$$P_e \approx \mathcal{Q}\Big(\frac{M}{\sqrt{MP_n}}\Big) = \mathcal{Q}\Big(\sqrt{\mathrm{SNR}}\Big), \qquad \mathcal{Q}(x) = \tfrac{1}{2}\operatorname{erfc}(x/\sqrt{2}). \tag{4}$$

Exact result. W_k is a sum of M independent uniform variables, whose density is the M-fold convolution of a rectangle (an Irwin–Hall density scaled to $[-Ma, Ma]$). Two consequences are useful checks: for $M = 1$, $P_e = (a-1)/(2a)$ for $a > 1$; and for any M, $P_e = 0$ when $a \le 1$ because $|W_k| \le Ma \le M$. Because the true density has bounded support, its tails fall off faster than the Gaussian, so (4) overestimates P_e at high SNR. The exact P_e was computed numerically by repeated convolution (`exact_ber_uniform.m`).

2.2 Sinusoidal jamming without spreading

With the jammer present, $y_k = Md_k + W_k + J_k$, where

$$J_k = A \sum_{n=kM}^{kM+M-1} \sin \omega_0 n = A \underbrace{\frac{\sin(M\omega_0/2)}{\sin(\omega_0/2)}}_{D_M(\omega_0)} \sin\Big(\omega_0\big(kM + \tfrac{M-1}{2}\big)\Big). \tag{5}$$

D_M is the Dirichlet kernel: the integrate-and-dump demodulator is a low-pass filter with gain M at DC, main lobe $|\omega_0| < 2\pi/M$ and nulls at $\omega_0 = 2\pi\ell/M$. Two regimes follow.

- **In-band jammer** ($\omega_0 \ll 2\pi/M$): $D_M \approx M$, so the jammer is integrated exactly like the signal, $|J_k|$ reaches AM, and an error occurs whenever $|A\sin(\cdot)| > 1$. Increasing M does *not* help.
- **Out-of-band jammer**: $|J_k| \le A/|\sin(\omega_0/2)|$, independent of M, so the demodulator already rejects it unless A is very large.

Averaging over the jammer phase ϕ at the start of a bit (uniform for incommensurate ω_0) and treating W_k as Gaussian,

$$P_e^{\text{no PN}} = \mathrm{E}_\phi\Big[\tfrac{1}{2}\mathcal{Q}\Big(\frac{M + J(\phi)}{\sqrt{MP_n}}\Big) + \tfrac{1}{2}\mathcal{Q}\Big(\frac{M - J(\phi)}{\sqrt{MP_n}}\Big)\Big]. \tag{6}$$

2.3 Direct-sequence spreading

The transmitter sends $s(n)p(n)$, where $p(n) = \pm 1$ are equiprobable i.i.d. chips. The receiver multiplies by the same $p(n)$; since $p^2(n) = 1$,

$$r(n)p(n) = s(n) + w(n)p(n) + A\sin(\omega_0 n)\,p(n). \tag{7}$$

The desired signal is restored exactly, the noise is statistically unchanged, and the jammer is multiplied by a random ± 1 sequence. Its spectrum is therefore spread uniformly over $(-\pi, \pi]$: the jammer becomes zero-mean white noise of power $A^2/2$. After summing over a bit,

$$J'_k = A\sum_n p(n)\sin\omega_0 n, \qquad \mathrm{E}[J'_k] = 0, \qquad \mathrm{Var}[J'_k] = A^2\sum_n \sin^2\omega_0 n \approx \frac{MA^2}{2}, \tag{8}$$

and the signal-to-interference-plus-noise ratio at the detector is

$$\mathrm{SINR} = \frac{M^2}{MP_n + MA^2/2} = \frac{M}{P_n + A^2/2}, \qquad P_e^{\mathrm{PN}} \approx \mathcal{Q}\left(\sqrt{\frac{M}{P_n + A^2/2}}\right). \tag{9}$$

Comparing with the per-sample signal-to-jammer ratio $2/A^2$ shows a *processing gain* $G_p = M$ ($10\log_{10} M$ dB): spreading lets the integrator average the jammer down in exactly the way it averages white noise. For very low ω_0 the factor $\sum\sin^2$ differs from bit to bit, so the simulations are compared with the phase-averaged form $\mathrm{E}_\phi[\mathcal{Q}(M/\sqrt{MP_n + A^2\sum\sin^2(\omega_0 n + \phi)})]$ (`ber_theory_jam.m`).

Consequence for the condition $A \geq M$. The jammer-to-signal power ratio is $\mathrm{JSR} = A^2/2$, so with $A = M$ it is $M^2/2$ while the processing gain is only M. Equation (9) then gives $\mathrm{SINR} \approx 2/M$, which *decreases* with M: under the brief's condition neither system can reach a low error rate, and the comparison must be read in terms of which system degrades less. This prediction is confirmed in Section 5.

3 MATLAB implementation

3.1 Structure

Each block of Figures 1 and 2 is a separate function in the `matlab/` folder (Table 1); the five functions required by Project 12.7 are reused unchanged in Project 12.8. The code uses only core MATLAB (no toolboxes) and also runs in GNU Octave, which was used to produce the results in this report. Full listings are in Appendix A.

Table 1: MATLAB functions and the block each implements.

Function	Block	Operation
`bit_generator(N)`	Binary data generator	$b_k = [u_k \geq 0.5]$, $u_k \sim \mathcal{U}(0,1)$
`modulator(b,M)`	Modulator	repeat $2b_k - 1$ for M samples
`noise_generator(L,a)`	Noise generator	$a(2u-1)$, uniform on $(-a, a)$
`demodulator(r,M)`	Demodulator	column sums of an $M \times N$ reshape
`detector(y,b)`	Detector & counter	$\hat{b}_k = [y_k > 0]$, count $\hat{b}_k \neq b_k$
`pn_generator(L)`	PN generator	equiprobable ± 1 chips
`sine_generator(L,A,w0,n0)`	Sine generator	$A\sin\omega_0 n$, continuous phase
`binary_comm_sim`	Whole system	chains the blocks, Monte-Carlo loop
`exact_ber_uniform`	Theory	exact P_e by numerical convolution
`ber_theory_jam`	Theory	Equations (6) and (9), phase averaged

The core of the simulator, `binary_comm_sim`, follows the block diagram line by line:

```
b = bit_generator(Nb); % binary data generator
s = modulator(b, M); % modulator
if use_pn
    p = pn_generator(L); % PN sequence generator
    s = s .* p; % spreading
end
r = s + noise_generator(L, a); % channel: wideband noise
if A ~= 0
    r = r + sine_generator(L, A, w0, n0); % channel: sine jammer
end
if use_pn
    r = r .* p; % despreading
end
y = demodulator(r, M); % demodulator
[~, e] = detector(y, b); % detector / error counter
```

3.2 Simulation design

- **Vectorisation and memory.** Bits are processed in blocks of up to 2×10^7 samples so that $M = 1000$ runs do not exhaust memory; the jammer sample index n_0 is carried between blocks so its phase is continuous.
- **Stopping rule.** For Project 12.7 each SNR point runs until 200 errors are counted or 2×10^7 bits are simulated. With 200 errors the relative standard error of the BER estimate is about $1/\sqrt{200} \approx 7\%$. For the $A = M$ experiment a fixed budget of 2×10^7 samples per point (4×10^5 bits at $M = 50$, 2×10^4 bits at $M = 1000$) was used; at BER ≈ 0.45 this gives a standard error below 0.004.
- **Parameters.** Unless stated otherwise $P_n = 1$ ($a = \sqrt{3}$). Two jammer frequencies were used: $\omega_0 = 0.002$ rad/sample, inside the demodulator pass-band $2\pi/M$ for every M tested, and $\omega_0 = 1$ rad/sample ($\approx 0.318\pi$), outside it. Both are deliberately incommensurate with $2\pi/M$; a frequency such as 0.1π with $M = 100$ places the jammer exactly on a null of D_M and gives a misleading zero error rate.
- **Reproducibility.** Seeds are fixed with `rng`; `run_all.m` regenerates every number and figure, and the tables in this report are read directly from the files the scripts write.

4 Results: Project 12.7

Figure 3 traces eight bits through the system at $M = 10$, $P_n = 1$ (SNR = 10 dB). The received samples are individually unreliable (many have the wrong sign), but summing M of them gives decision statistics close to ± 10 and all eight bits are detected correctly.

The BER was measured in the two ways suggested in the brief: (a) $P_n = 1$ fixed with $M = 1, \ldots, 20$, and (b) $M = 10$ fixed with P_n varied to give SNR = 0–13 dB. Figure 4 and Table 2 show the results.

Observations:

1. The BER falls steeply with SNR, from about 0.2 at 0 dB to 10^{-6} near 13 dB; each extra dB costs roughly a factor of three to ten in BER at high SNR.
2. The simulation agrees with the exact result at every point within the expected Monte-Carlo scatter, which validates the implementation of all five blocks. At $M = 1$ the simulated 0.21 matches $(a - 1)/(2a) = 0.211$.
3. The Gaussian approximation is accurate between about 3 and 8 dB but is optimistic at $M = 1$ (a single uniform sample has heavier mass near its edges than a Gaussian) and increasingly pessimistic at high SNR, by a factor of three at 13 dB in sweep (a).

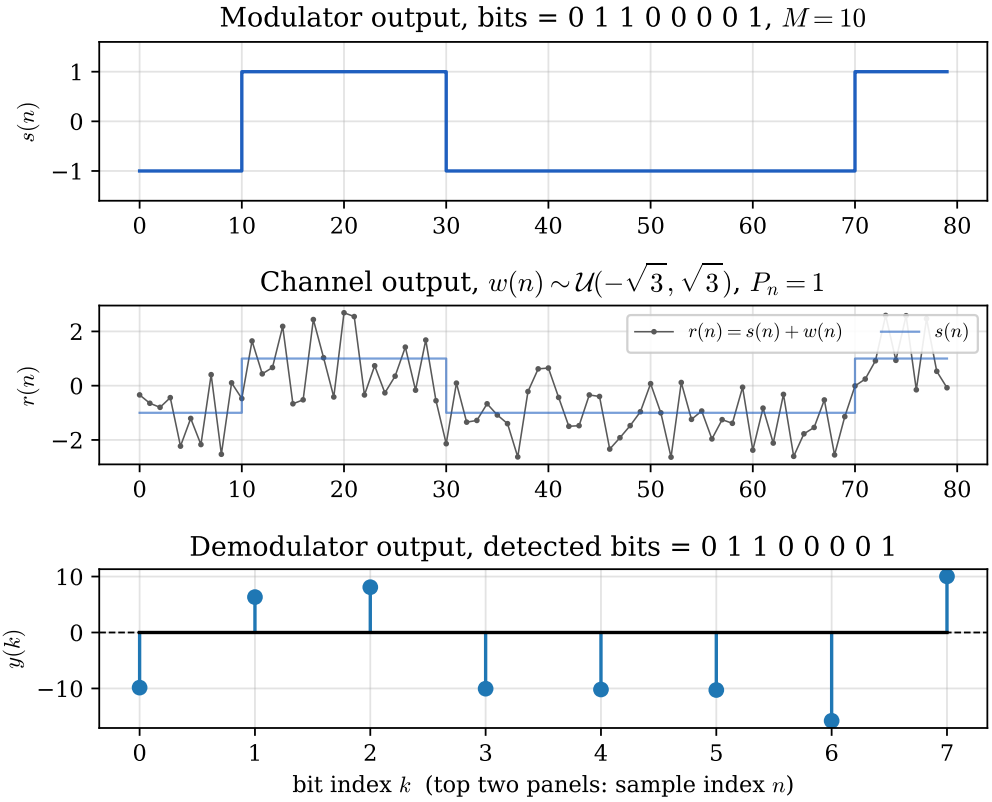


Figure 3: Signals at the modulator output, channel output and demodulator output for $M = 10$ and $P_n = 1$.

4. The two sweeps coincide at low SNR, as the definition SNR $= M/P_n$ implies, but separate at high SNR: for sweep (b) with $M = 10$ fixed, high SNR means a small noise range a, and as $a \to 1$ the maximum possible noise sum Ma approaches M, so $P_e \to 0$ much faster than the Gaussian tail. Only one error in 2×10^7 bits was observed at 13 dB, consistent with the exact value 1.3×10^{-7}. The SNR alone does not fully describe the system when the noise is not Gaussian.

5 Results: Project 12.8

5.1 The brief's experiment: $A = M$

Table 3 and Figure 5 give the BER with and without the PN sequence for $M = 50, 100, 500, 1000$ with the smallest jammer amplitude the brief allows, $A = M$.

- **In-band jammer.** Without PN the BER is 0.49–0.50 for every M: the jammer passes through the integrator with the full gain M, so the bit sum is dominated by $AM \sin(\cdot)$ and the decision is essentially a coin toss. With PN the BER is lower, 0.37 at $M = 50$, because the jammer has been converted into noise whose bit-sum standard deviation is only $A\sqrt{M/2}$. The improvement shrinks as M grows because SINR $\approx 2/M$ when $A = M$.
- **Out-of-band jammer.** Without PN the demodulator rejects the jammer through its $\sin(\omega_0/2)$ roll-off. The error rate depends only on whether $A|D_M(1)|$ exceeds M: it does not for $M = 50, 100$ (BER $= 0$), it does for $M = 500$ ($|D_M|/M = 2.02$, BER $= 0.33$) and only just does for $M = 1000$ ($|D_M|/M = 0.98$, BER $= 0.013$). With PN the BER is 0.42–0.48 regardless of ω_0 and is therefore *worse* here: spreading takes a jammer the receiver was already rejecting and turns it into in-band noise.

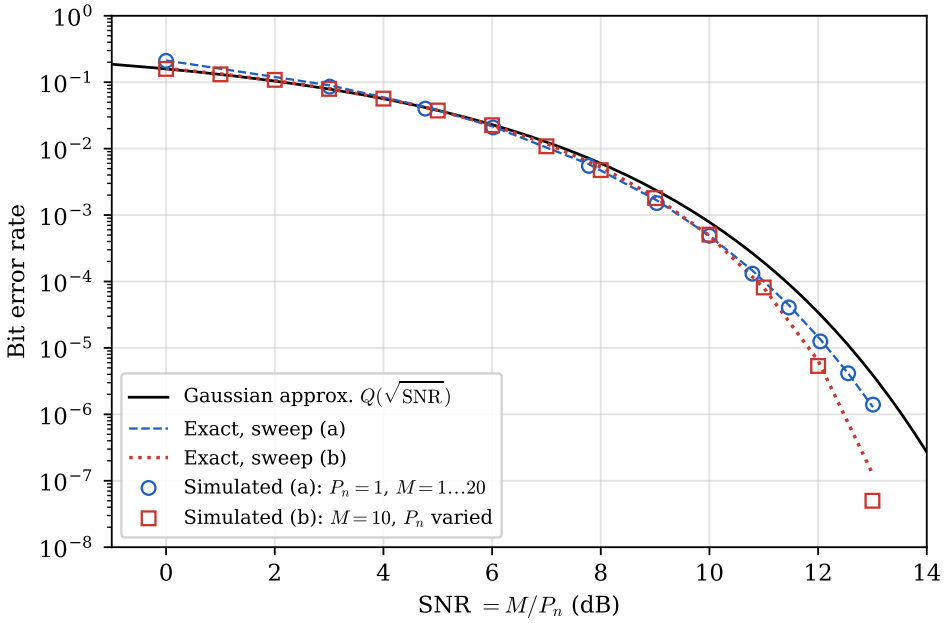


Figure 4: BER against SNR $= M/P_n$ for the binary system with uniform noise.

Table 2: Sweep (a): $P_n = 1$, M varied. Simulated BER compared with the exact probability and the Gaussian approximation $\mathcal{Q}(\sqrt{\text{SNR}})$.

M	SNR (dB)	Simulated	Exact	$\mathcal{Q}(\sqrt{\text{SNR}})$	Bits
1	0.00	0.2095	0.2122	0.1587	20000
2	3.01	0.0858	0.0898	0.0786	20000
3	4.77	0.0400	0.0428	0.0416	20000
4	6.02	0.0208	0.0215	0.0228	20000
6	7.78	5.50×10^{-3}	5.83×10^{-3}	7.15×10^{-3}	40000
8	9.03	1.51×10^{-3}	1.67×10^{-3}	2.34×10^{-3}	140000
10	10.00	4.88×10^{-4}	4.89×10^{-4}	7.83×10^{-4}	420000
12	10.79	1.31×10^{-4}	1.46×10^{-4}	2.66×10^{-4}	1540000
14	11.46	4.07×10^{-5}	4.41×10^{-5}	9.14×10^{-5}	4940000
16	12.04	1.25×10^{-5}	1.34×10^{-5}	3.17×10^{-5}	15960000
18	12.55	4.15×10^{-6}	4.15×10^{-6}	1.10×10^{-5}	20000000
20	13.01	1.40×10^{-6}	1.28×10^{-6}	3.87×10^{-6}	20000000

- The simulated and theoretical values agree to within ± 0.007 in every cell, so the behaviour is a property of the system, not of the simulation.

5.2 Effect of jammer power

To see the processing gain directly, the in-band jammer amplitude was swept from $A = 0.1$ to 1000 (Figure 6), and the jammer-to-signal ratio at which each system reaches BER $= 10^{-3}$ was read from the theory curves (Table 4).

Without PN every curve breaks at roughly the same point, JSR ≈ -3 to 0 dB, i.e. $A \approx 1$: once the jammer amplitude exceeds the signal amplitude, errors occur no matter how many samples are integrated. With PN the curves move right by roughly $10 \log_{10} M$, and the improvement in Table 4 grows from 11 dB at $M = 50$ to 20 dB at $M = 1000$. The improvement is below the ideal $10 \log_{10} M$ because a jammer this slow does not average its $\sin^2$ within one bit, so some bits see more interference than others, and because the noise still contributes. The last column shows why the $A = M$ experiment saturates: the jammer is 26 dB to 37 dB stronger than the PN system can

Table 3: BER with the jammer amplitude $A = M$ and $P_n = 1$. Each entry is *simulated (theory)*, with theory from Equations (6) and (9).

		In-band, $\omega_0 = 0.002$		Out-of-band, $\omega_0 = 1$	
M	A	No PN	PN	No PN	PN
50	50	0.4932 (0.4934)	0.3744 (0.3681)	0.0000 (0.0000)	0.4208 (0.4208)
100	100	0.4947 (0.4971)	0.4049 (0.4021)	0.0000 (0.0000)	0.4451 (0.4438)
500	500	0.4963 (0.5000)	0.4673 (0.4687)	0.3341 (0.3355)	0.4768 (0.4748)
1000	1000	0.4975 (0.5000)	0.4757 (0.4814)	0.0126 (0.0124)	0.4845 (0.4822)

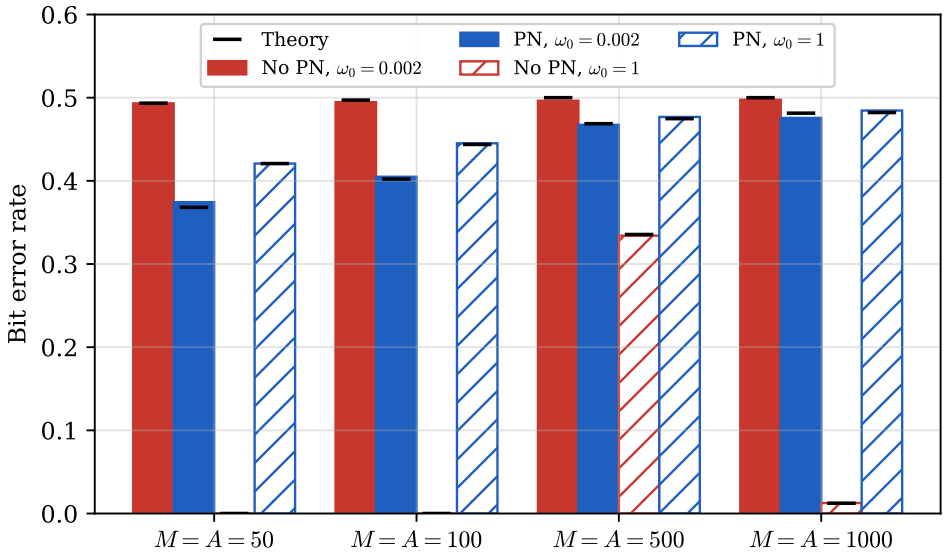


Figure 5: BER at $A = M$; black ticks mark the theoretical values.

tolerate.

5.3 Effect of jammer frequency

Figure 7 sweeps ω_0 over $(0, \pi]$ for $M = 100$. With PN the BER is flat in frequency, and it agrees with Equation (9): 0.44 at $A = 100$, and $\mathcal{Q}(\sqrt{100/13.5}) = 3.2 \times 10^{-3}$ at $A = 5$. Without PN the BER follows the Dirichlet kernel $|D_M(\omega_0)|$: it is 0.5 near DC, falls to zero at every null $\omega_0 = 2\pi\ell/M$, and decays with ω_0. At $A = 5$ the system without PN fails only for $\omega_0 < 0.03\pi$, i.e. inside the main lobe of the demodulator response.

5.4 Spectral view

Figure 8 shows the power spectral densities before and after despreading for $M = 100$, $A = 10$, $\omega_0 = 0.2\pi$. In the channel the spread signal is white with power 1 (0 dB), as is the noise, while the jammer is a line 40 dB above them. After despreading the roles swap: the desired signal collapses back into the narrow sinc^2 spectrum of a rectangular pulse, with 20 dB ($= 10 \log_{10} M$) at DC, and the jammer becomes a white floor at $10 \log_{10}(A^2/2) = 17$ dB. The integrate-and-dump demodulator passes only the band $|\omega| \lesssim 2\pi/M$, which contains all of the signal energy but only a fraction of order $1/M$ of the spread jammer power.

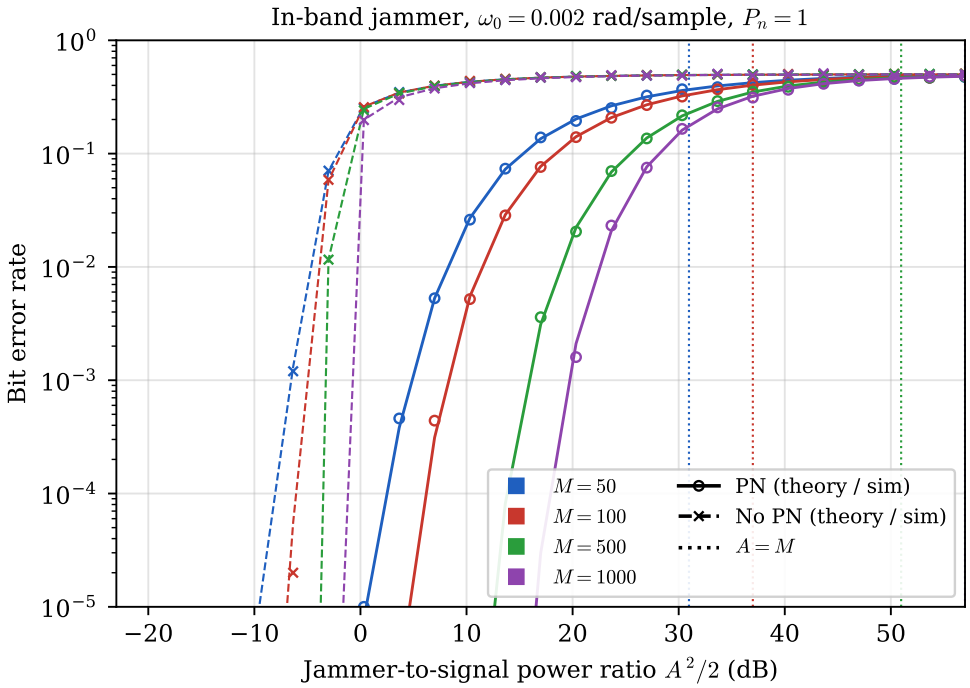


Figure 6: BER against jammer-to-signal power ratio $A^2/2$ for the in-band jammer. Solid lines and circles: PN; dashed lines and crosses: no PN. Dotted verticals mark $A = M$ for each M.

Table 4: Jammer-to-signal ratio (dB) tolerated at BER $= 10^{-3}$, in-band jammer, $P_n = 1$, and the ratio implied by the brief's condition $A = M$.

M	$10 \log_{10} M$	No PN	PN	Improvement	JSR at $A = M$
50	17.0	-6.5	4.9	11.4	31.0
100	20.0	-5.0	8.3	13.3	37.0
500	27.0	-3.3	16.0	19.2	51.0
1000	30.0	-0.7	19.7	20.5	57.0

6 Discussion

6.1 Effect of the PN sequence on the sinusoidal interference

Multiplying by a random ±1 sequence is a modulation by a wideband signal. In the frequency domain the jammer spectrum, a pair of lines at $\pm\omega_0$, is convolved with the flat spectrum of $p(n)$, so the receiver's despreading multiplier turns the sinusoid into white noise of the same power $A^2/2$ (Figure 8). The same operation applied to the desired signal, which was spread by the same $p(n)$, undoes the spreading because $p^2(n) = 1$. The PN sequence therefore acts as a key that concentrates only the signal back into the narrow band of the demodulator.

6.2 Why the spread-spectrum system outperforms the conventional one

1. **Processing gain.** The demodulator sums M samples. The desired signal adds coherently (amplitude M, power M^2), but the despread jammer adds incoherently (power $MA^2/2$), giving a gain of M in signal-to-jammer ratio. Without PN, an in-band jammer adds coherently as well, so integration gives no gain at all; this is why the conventional system fails at $A \approx 1$ whatever M is, while the PN system tolerates jammers 11 dB to 20 dB stronger (Table 4).

2. **Independence from the jammer frequency.** The PN system's performance depends only on the jammer power, not on ω_0 (Figure 7). A jammer cannot find a frequency where the

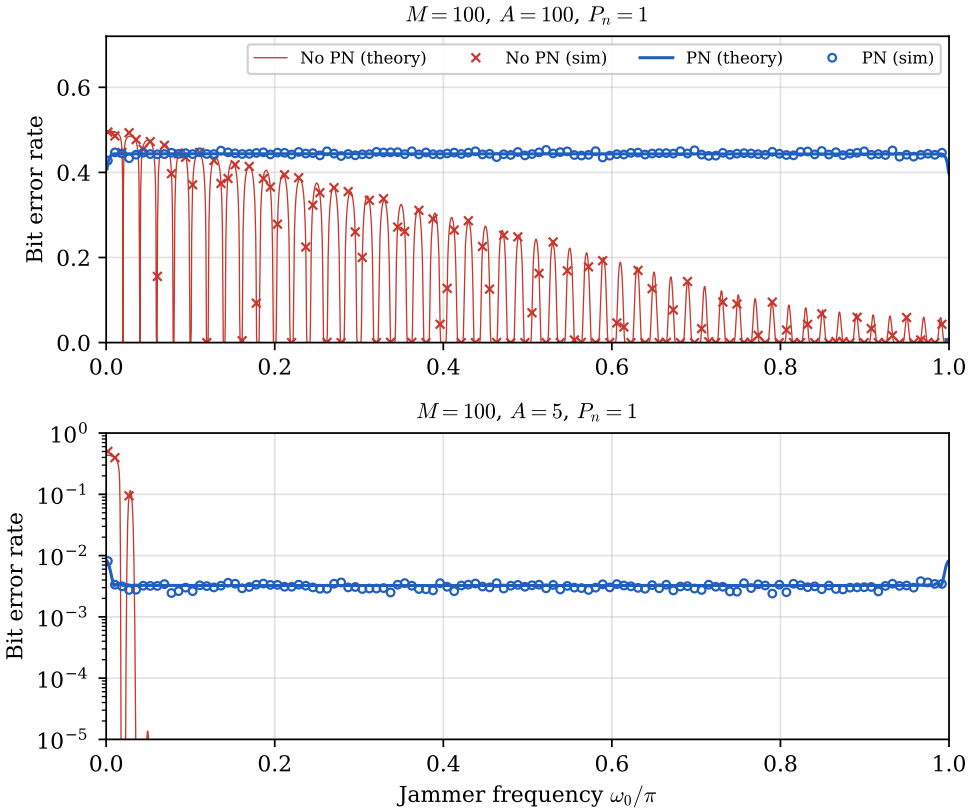


Figure 7: BER against jammer frequency for $M = 100$: $A = M$ (top, linear scale) and $A = 5$ (bottom, log scale).

receiver is weak; its best strategy is simply more power. In the conventional system the worst case, a jammer placed inside the signal band, is also the easiest for an adversary to choose.

3. **Low probability of interception.** Although not measured here, the spread signal has a flat spectrum at the noise level (Figure 8), which makes it difficult to detect or to target, and different PN codes allow several users to share the band (CDMA).

6.3 Interpreting the condition $A \geq M$

The brief asks for $A \geq M$. As derived in Section 2 and confirmed by Table 3, this places the jammer 31 dB to 57 dB above the signal, beyond the $10 \log_{10} M = 17$ dB to 30 dB processing gain, so neither system works reliably and the PN system's advantage appears only as a lower error rate against an in-band jammer (0.37 against 0.49 at $M = 50$). We also found that for an out-of-band jammer the conventional system can be better, because its integrator already rejects the tone. Rather than suggesting that spread spectrum is ineffective, this shows that the benefit is a fixed number of decibels: to obtain a low error rate with $A = M$ the processing gain would need to grow as M^2, which this system's single-PN-chip-per-sample structure cannot provide. Practical systems such as GPS obtain their 43 dB processing gain from a chip rate 2×10^4 times the bit rate.

6.4 Limitations

- Perfect bit and code synchronisation was assumed; in practice code acquisition and tracking are a significant part of a DSSS receiver.
- The PN sequence is generated by a random number generator, as the brief allows. Real systems use deterministic maximal-length or Gold sequences, whose autocorrelation and cross-correlation properties are guaranteed.

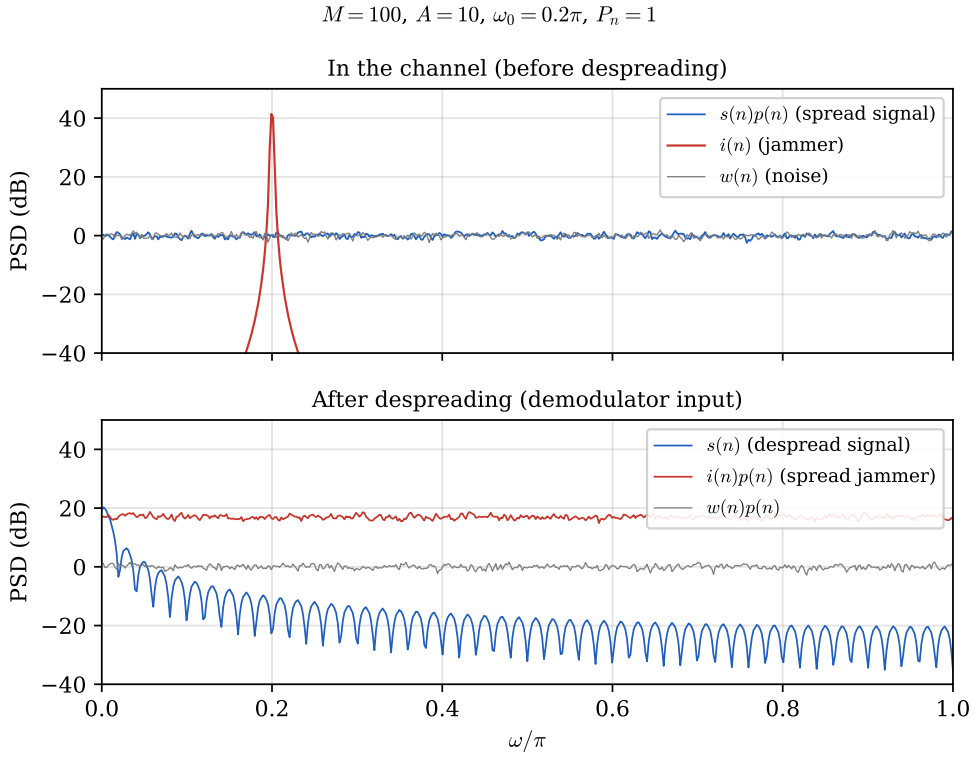


Figure 8: Power spectral densities (Hann-windowed averaged periodogram) in the channel and after despreading.

- Uniform noise was used as specified; a thermal-noise channel would be Gaussian, for which Equation (4) is exact.
- The Gaussian treatment of the despread jammer in Equation (9) is accurate for large M; at $M = 50$ the simulation and theory still agree within 0.007.

7 Conclusion

Both textbook systems were implemented in MATLAB as one function per block and validated against analytical results. For the binary system with uniform noise the BER follows the exact M-fold-convolution result over seven decades, and the familiar $\mathcal{Q}(\sqrt{\mathrm{SNR}})$ approximation is pessimistic at high SNR because the noise is bounded. For the spread-spectrum system, the PN sequence converts a narrowband jammer into wideband noise, which the integrate-and-dump demodulator then averages down by the processing gain M. This made the receiver tolerate in-band jammers 11 dB to 20 dB stronger than the conventional receiver and made its performance independent of the jammer frequency. Under the brief's condition $A = M$ the jammer power exceeds the processing gain by 14 dB to 27 dB, so both systems operate near BER = 0.5; the PN system still has the lower error rate against an in-band jammer, but not against an out-of-band one, which the conventional demodulator already filters out.

Individual contribution

[Describe your own contribution to the group work here: for example which blocks you implemented, which experiments you ran, and which parts of the analysis you led.]

References

[1] V. K. Ingle and J. G. Proakis, *Digital Signal Processing Using MATLAB*, 4th ed. Boston, MA: Cengage Learning, 2017, Projects 12.7 and 12.8.

[2] J. G. Proakis and M. Salehi, *Digital Communications*, 5th ed. New York: McGraw-Hill, 2008, ch. 12 (spread-spectrum signals).

[3] B. Sklar, *Digital Communications: Fundamentals and Applications*, 2nd ed. Upper Saddle River, NJ: Prentice Hall, 2001, ch. 12.

[4] U.S. Space Force, *Interface Specification IS-GPS-200*, NAVSTAR GPS Space Segment/Navigation User Segment Interfaces.

A MATLAB code

Block functions

```
function b = bit_generator(N)
% BIT_GENERATOR Binary data generator (Project 12.7, block 1).
% b = bit_generator(N) returns a 1xN row vector of independent,
% equiprobable binary digits (0 or 1).
    b = double(rand(1, N) >= 0.5);
end
```

Listing 1: bit_generator.m

```
function s = modulator(b, M)
% MODULATOR Rectangular-pulse binary modulator (Project 12.7, block 2).
% s = modulator(b, M) maps every bit 1 to M consecutive +1's and every
% bit 0 to M consecutive -1's, i.e. a sampled antipodal rectangular pulse.
% Output length is M*length(b).
    a = 2*b(:).' - 1; % 0 -> -1, 1 -> +1
    s = reshape(repmat(a, M, 1), 1, []);
end
```

Listing 2: modulator.m

```
function w = noise_generator(L, a)
% NOISE_GENERATOR Uniform additive noise (Project 12.7, block 3).
% w = noise_generator(L, a) returns L i.i.d. samples uniformly
% distributed on (-a, a). The noise power is Pn = a^2/3.
    w = a * (2*rand(1, L) - 1);
end
```

Listing 3: noise_generator.m

```
function y = demodulator(r, M)
% DEMODULATOR Integrate-and-dump demodulator (Project 12.7, block 4).
% y = demodulator(r, M) sums each block of M successive received
% samples. The receiver is assumed to be bit-synchronised, so block k
% is r((k-1)*M+1 : k*M). Returns one decision statistic per bit.
    y = sum(reshape(r, M, []), 1);
end
```

Listing 4: demodulator.m

```
function [b_hat, nerr] = detector(y, b)
% DETECTOR Threshold detector and error counter (Project 12.7, block 5).
% [b_hat, nerr] = detector(y, b) decides 1 when y > 0 and 0 otherwise,
% then counts the decisions that disagree with the transmitted bits b.
```

```
% (y == 0 has probability zero for continuous noise.)
    b_hat = double(y > 0);
    nerr = sum(b_hat ~= b);
end
```

Listing 5: detector.m

```
function p = pn_generator(L)
% PN_GENERATOR Binary (+/-1) pseudo-noise sequence (Project 12.8).
% p = pn_generator(L) returns L equiprobable +/-1 chips generated with a
% random number generator. The receiver uses the identical sequence.
    p = 2*double(rand(1, L) >= 0.5) - 1;
end
```

Listing 6: pn_generator.m

```
function i_n = sine_generator(L, A, w0, n0)
% SINE_GENERATOR Sinusoidal interference i(n) = A sin(w0 n) (Project 12.8).
% i_n = sine_generator(L, A, w0) returns L samples starting at n = 0.
% i_n = sine_generator(L, A, w0, n0) starts at n = n0, so a long run can
% be generated in blocks without breaking the phase of the jammer.
    if nargin < 4, n0 = 0; end
    n = n0 + (0:L-1);
    i_n = A * sin(w0 * n);
end
```

Listing 7: sine_generator.m

System simulator and theory

```
function [ber, nerr, nbits] = binary_comm_sim(M, a, opts)
% BINARY_COMM_SIM Monte-Carlo run of the systems in Figs 12.16 and 12.18.
% [ber, nerr, nbits] = binary_comm_sim(M, a, opts)
% M samples per bit
% a uniform noise half-width (noise on (-a,a), Pn = a^2/3)
% opts struct with optional fields
% .use_pn true to spread with a +/-1 PN sequence (default false)
% .A sinusoidal interference amplitude (default 0)
% .w0 interference frequency in rad/sample (default 0)
% .min_err stop once this many errors are seen (default 200)
% .max_bits hard limit on simulated bits (default 1e6)
% .block bits simulated per block (default 2e4)
% Bits are simulated in blocks so large M does not exhaust memory; the
% jammer phase runs continuously across blocks.
    if nargin < 3, opts = struct(); end
    use_pn = getopt(opts, 'use_pn', false);
    A = getopt(opts, 'A', 0);
    w0 = getopt(opts, 'w0', 0);
    min_err = getopt(opts, 'min_err', 200);
    max_bits = getopt(opts, 'max_bits', 1e6);
    block = getopt(opts, 'block', 2e4);
    % keep each block below ~2e7 samples
    block = max(1, min(block, floor(2e7 / M)));

    nerr = 0; nbits = 0; n0 = 0;
    while nerr < min_err && nbits < max_bits
        Nb = min(block, max_bits - nbits);
        L = Nb * M;
        b = bit_generator(Nb); % binary data generator
        s = modulator(b, M); % modulator
        if use_pn
            p = pn_generator(L); % PN sequence generator
            s = s .* p; % spreading
```

```
        end
        r = s + noise_generator(L, a); % channel: wideband noise
        if A ~= 0
            r = r + sine_generator(L, A, w0, n0); % channel: sine jammer
        end
        if use_pn
            r = r .* p; % despreading
        end
        y = demodulator(r, M); % demodulator
        [~, e] = detector(y, b); % detector / error counter
        nerr = nerr + e; nbits = nbits + Nb; n0 = n0 + L;
    end
    ber = nerr / nbits;
end

function v = getopt(s, f, d)
    if isfield(s, f), v = s.(f); else, v = d; end
end
```

Listing 8: binary_comm_sim.m

```
function pe = exact_ber_uniform(M, a)
% EXACT_BER_UNIFORM Exact bit error probability for Project 12.7.
% pe = exact_ber_uniform(M, a) returns P(W > M), where W is the sum of M
% i.i.d. Uniform(-a,a) noise samples. By symmetry this is the bit error
% probability of the integrate-and-dump receiver for either bit value.
% The pdf of W is obtained by numerically convolving M uniform pdfs.
% Works for scalar M and vector a.
    pe = zeros(size(a));
    for k = 1:numel(a)
        ak = a(k);
        if ak <= 1 % |W| <= M*a <= M: no error possible
            pe(k) = 0; continue;
        end
        dx = ak / 400; % grid spacing
        x1 = -ak:dx:ak;
        f1 = ones(size(x1)) / (2*ak); % Uniform(-a,a) pdf
        f = f1;
        for m = 2:M
            f = conv(f, f1) * dx;
        end
        x = (0:numel(f)-1) * dx - M*ak; % support of the M-fold sum
        f = f / (sum(f) * dx); % remove discretisation drift
        pe(k) = sum(f(x > M)) * dx;
    end
end
```

Listing 9: exact_ber_uniform.m

```
function [ber_nopn, ber_pn] = ber_theory_jam(M, A, w0, Pn)
% BER_THEORY_JAM Analytical BER with sinusoidal jamming (Project 12.8).
% [ber_nopn, ber_pn] = ber_theory_jam(M, A, w0, Pn) averages over the
% jammer phase phi seen at the start of a bit (uniform on [0, 2pi)).
% The summed noise is treated as Gaussian with variance M*Pn (CLT).
%
% Without PN the jammer adds a deterministic offset to the bit sum,
% J(phi) = A * sum_{n=0}^{M-1} sin(w0 n + phi),
% so Pe(phi) = [Q((M+J)/s) + Q((M-J)/s)] / 2, s = sqrt(M Pn).
%
% With PN the jammer is multiplied by i.i.d. +/-1 chips, so its sum is
% zero mean with variance A^2 * sum sin^2(w0 n + phi), giving
% Pe(phi) = Q( M / sqrt(M Pn + A^2 sum sin^2(w0 n + phi)) ).
% Scalar M, w0, Pn; A may be a vector.
    phi = 2*pi*(0:359)/360;
```

```
    n = (0:M-1).';
    S = sin(w0*n + phi); % M x 360
    J1 = sum(S, 1); % sum sin per phase
    J2 = sum(S.^2, 1); % sum sin^2 per phase
    sig = sqrt(M*Pn);
    ber_nopn = zeros(size(A)); ber_pn = ber_nopn;
    for k = 1:numel(A)
        J = A(k) * J1;
        ber_nopn(k) = mean(0.5*(qfunc_local((M + J)/sig) + qfunc_local((M - J)/sig)));
        ber_pn(k) = mean(qfunc_local(M ./ sqrt(M*Pn + A(k)^2 * J2)));
    end
end
```

Listing 10: ber_theory_jam.m

Experiment scripts

```
%% Project 12.7: Binary data communications system (Ingle & Proakis 4e)
% Simulates the system of Fig. 12.16:
% bit generator -> modulator (M samples/bit) -> + uniform noise ->
% integrate-and-dump demodulator -> detector & error counter
% and measures the bit error rate (BER) against SNR = M / Pn, Pn = a^2/3.
%
% Two sweeps are run, as suggested in the brief:
% (a) Pn fixed (= 1) while M is changed,
% (b) M fixed (= 10) while Pn (i.e. a) is changed.
% Results are compared with the Gaussian (CLT) approximation
% Pe ~ Q(sqrt(M/Pn)) and with the exact result from exact_ber_uniform.
clear; close all; clc;
rng(12); % reproducible results
if exist('OCTAVE_VERSION', 'builtin'), graphics_toolkit('gnuplot'); end

min_err = 200; % stop after this many errors
max_bits = 2e7; % ... or this many bits

%% Illustration of the signals through each block
M = 10; a = sqrt(3); % Pn = 1, SNR = 10 dB
b = bit_generator(8);
s = modulator(b, M);
w = noise_generator(numel(s), a);
r = s + w;
y = demodulator(r, M);
b_hat = detector(y, b);
n = 0:numel(s)-1;

f = figure('visible', 'off');
subplot(3,1,1); stairs(n, s, 'LineWidth', 1.2); ylim([-1.5 1.5]); grid on;
title(sprintf('Modulator output s(n), bits = [%s]', num2str(b)));
ylabel('s(n)');
subplot(3,1,2); plot(n, r, '.-'); grid on;
title(sprintf('Received r(n) = s(n) + w(n), a = %.2f (P_n = %.2f)', a, a^2/3));
ylabel('r(n)');
subplot(3,1,3); stem(0:numel(y)-1, y, 'filled'); hold on;
plot([-0.5 numel(y)-0.5], [0 0], 'k--'); grid on;
title(sprintf('Demodulator output y(k), detected bits = [%s]', num2str(b_hat)));
xlabel('bit index k'); ylabel('y(k)');
save_fig(f, 'p127_waveforms');

%% (a) Pn fixed, M varied
a_fix = sqrt(3); Pn_fix = a_fix^2/3; % Pn = 1
M_list = [1 2 3 4 6 8 10 12 14 16 18 20];
ber_a = zeros(size(M_list)); nb_a = ber_a;
for k = 1:numel(M_list)
```

```
    [ber_a(k), ~, nb_a(k)] = binary_comm_sim(M_list(k), a_fix, ...
        struct('min_err', min_err, 'max_bits', max_bits));
    fprintf('(a) M = %2d SNR = %5.2f dB BER = %.3e (%d bits)\n', ...
        M_list(k), 10*log10(M_list(k)/Pn_fix), ber_a(k), nb_a(k));
end
snr_a = M_list / Pn_fix;
gauss_a = qfunc_local(sqrt(snr_a));
exact_a = zeros(size(M_list));
for k = 1:numel(M_list), exact_a(k) = exact_ber_uniform(M_list(k), a_fix); end

%% (b) M fixed, Pn varied
M_fix = 10;
snr_db = 0:1:13;
Pn_b = M_fix ./ 10.^(snr_db/10);
a_b = sqrt(3*Pn_b);
ber_b = zeros(size(snr_db)); nb_b = ber_b;
for k = 1:numel(snr_db)
    [ber_b(k), ~, nb_b(k)] = binary_comm_sim(M_fix, a_b(k), ...
        struct('min_err', min_err, 'max_bits', max_bits));
    fprintf('(b) a = %5.2f SNR = %5.2f dB BER = %.3e (%d bits)\n', ...
        a_b(k), snr_db(k), ber_b(k), nb_b(k));
end
gauss_b = qfunc_local(sqrt(10.^(snr_db/10)));
exact_b = exact_ber_uniform(M_fix, a_b);

%% Plot BER vs SNR for both sweeps
snr_fine = linspace(-1, 14, 200);
f = figure('visible', 'off');
semilogy(10*log10(snr_a), ber_a, 'o', 'MarkerSize', 7, 'LineWidth', 1.2); hold on;
semilogy(snr_db, ber_b, 's', 'MarkerSize', 7, 'LineWidth', 1.2);
semilogy(snr_fine, qfunc_local(sqrt(10.^(snr_fine/10))), 'k-', 'LineWidth', 1.2);
semilogy(10*log10(snr_a), exact_a, 'b--', 'LineWidth', 1);
semilogy(snr_db, exact_b, 'r:', 'LineWidth', 1.4);
grid on; xlim([-1 14]); ylim([1e-8 1]);
xlabel('SNR = M / P_n (dB)'); ylabel('Bit error rate');
title('Project 12.7: BER of the binary system with uniform noise');
legend('Sim. (a): P_n = 1, M varied', 'Sim. (b): M = 10, P_n varied', ...
       'Gaussian approx. Q(\surd SNR)', 'Exact (a)', 'Exact (b)', ...
       'Location', 'southwest');
save_fig(f, 'p127_ber');

%% Save numeric results
res127 = struct('M_list', M_list, 'snr_a_db', 10*log10(snr_a), 'ber_a', ber_a, ...
    'nbits_a', nb_a, 'exact_a', exact_a, 'gauss_a', gauss_a, ...
    'M_fix', M_fix, 'snr_b_db', snr_db, 'a_b', a_b, 'ber_b', ber_b, ...
    'nbits_b', nb_b, 'exact_b', exact_b, 'gauss_b', gauss_b, ...
    'wave_b', b, 'wave_s', s, 'wave_r', r, 'wave_y', y, 'wave_bhat', b_hat, 'wave_M', 10);
save(fullfile(fileparts(mfilename('fullpath')), '..', 'results', 'res127.mat'), 'res127', '-v7');
write_table_127(res127);
```

Listing 11: project12_7.m

```
%% Project 12.8: Binary spread-spectrum communications (Ingle & Proakis 4e)
% Simulates the system of Fig. 12.18: the Project 12.7 system with the
% modulator output multiplied by a +/-1 PN sequence p(n), a channel that
% adds wideband noise w(n) and a sinusoidal jammer i(n) = A sin(w0 n),
% and a receiver that multiplies by the same p(n) before demodulating.
%
% Experiments
% 1. Brief: BER with and without PN for M = 50, 100, 500, 1000, A = M,
% for an in-band (low-frequency) and an out-of-band jammer.
% 2. BER against jammer amplitude A (jammer-to-signal ratio) per M.
% 3. BER against jammer frequency w0 for fixed M.
% 4. Power spectra before and after despreading.
clear; close all; clc;
```

```
rng(128);
if exist('OCTAVE_VERSION', 'builtin'), graphics_toolkit('gnuplot'); end

a = sqrt(3); Pn = a^2/3; % wideband noise, Pn = 1
M_list = [50 100 500 1000];
w_in = 0.002; % rad/sample, inside 2*pi/M for all M
w_out = 1; % rad/sample (~0.318*pi)
min_err = Inf; % experiment 1: fixed bit budget
bits_for = @(M) max(2e4, round(2e7/M)); % ~2e7 samples per point

%% 1. Brief experiment: A = M
fprintf('Experiment 1: A = M, Pn = %.2f\n', Pn);
w_set = [w_in w_out];
T = zeros(numel(M_list), 2*numel(w_set)); % simulated BERs
Tth = T; % theory
for i = 1:numel(M_list)
    M = M_list(i); A = M;
    for j = 1:numel(w_set)
        o = struct('A', A, 'w0', w_set(j), 'min_err', min_err, 'max_bits', bits_for(M));
        o.use_pn = false; ber0 = binary_comm_sim(M, a, o);
        o.use_pn = true; ber1 = binary_comm_sim(M, a, o);
        [t0, t1] = ber_theory_jam(M, A, w_set(j), Pn);
        T(i, 2*j-1:2*j) = [ber0 ber1];
        Tth(i, 2*j-1:2*j) = [t0 t1];
        fprintf(' M = %4d w0 = %.3f no PN: %.4f (th %.4f) PN: %.4f (th %.4f)\n', ...
            M, w_set(j), ber0, t0, ber1, t1);
    end
end

f = figure('visible', 'off');
semilogx(M_list, T(:,1), 'ro-', M_list, T(:,2), 'bo-', ...
         M_list, T(:,3), 'rs--', M_list, T(:,4), 'bs--', 'LineWidth', 1.2, 'MarkerSize', 7);
grid on; ylim([0 0.6]); xlim([40 1200]);
xlabel('Samples per bit M (with A = M)'); ylabel('Bit error rate');
title('Project 12.8: BER with jammer amplitude A = M, P_n = 1');
legend(sprintf('No PN, \\omega_0 = %.3f', w_in), sprintf('PN, \\omega_0 = %.3f', w_in), ...
       sprintf('No PN, \\omega_0 = %g', w_out), sprintf('PN, \\omega_0 = %g', w_out), ...
       'Location', 'southwest');
save_fig(f, 'p128_ber_A_eq_M');

%% 2. BER against jammer amplitude (in-band jammer)
A_list = logspace(-1, 3, 25);
jsr_db = 10*log10(A_list.^2/2); % jammer power / signal power per sample
B0 = nan(numel(M_list), numel(A_list)); B1 = B0; Th0 = B0; Th1 = B0;
for i = 1:numel(M_list)
    M = M_list(i);
    [Th0(i,:), Th1(i,:)] = ber_theory_jam(M, A_list, w_in, Pn);
    for k = 1:numel(A_list)
        o = struct('A', A_list(k), 'w0', w_in, 'min_err', 1000, ...
                   'max_bits', round(bits_for(M)/4));
        o.use_pn = false; B0(i,k) = binary_comm_sim(M, a, o);
        o.use_pn = true; B1(i,k) = binary_comm_sim(M, a, o);
    end
    fprintf('Experiment 2: M = %d done\n', M);
end

f = figure('visible', 'off');
cols = {'b', 'r', [0 0.6 0], 'm'};
h = zeros(1, numel(M_list));
for i = 1:numel(M_list)
    h(i) = semilogy(jsr_db, max(B1(i,:), 1e-6), 'o', 'Color', cols{i}, 'MarkerSize', 5); hold on;
    semilogy(jsr_db, max(Th1(i,:), 1e-6), '-', 'Color', cols{i}, 'LineWidth', 1.1);
    semilogy(jsr_db, max(B0(i,:), 1e-6), 'x', 'Color', cols{i}, 'MarkerSize', 6);
    semilogy(jsr_db, max(Th0(i,:), 1e-6), '--', 'Color', cols{i}, 'LineWidth', 0.8);
```

```
    jA = 10*log10(M_list(i)^2/2); % A = M for this M
    semilogy([jA jA], [1e-6 1], '-.', 'Color', cols{i}, 'LineWidth', 0.6);
end
grid on; ylim([1e-5 1]); xlim([min(jsr_db) max(jsr_db)]);
xlabel('Jammer-to-signal power ratio A^2/2 (dB)'); ylabel('Bit error rate');
title(sprintf('BER vs jammer power, \\omega_0 = %.3f rad/sample (o PN, x no PN)', w_in));
legend(h, arrayfun(@(m) sprintf('M = %d', m), M_list, 'UniformOutput', false), ...
       'Location', 'southeast');
save_fig(f, 'p128_ber_vs_jsr');

%% 3. BER against jammer frequency
M = 100;
w_list = pi * linspace(0.002, 1, 120); % simulated points
w_fine = pi * linspace(0.001, 1, 2000); % theory curve
A_pair = [M 5];
W0 = zeros(2, numel(w_list)); W1 = W0;
Wt0 = zeros(2, numel(w_fine)); Wt1 = Wt0;
for q = 1:2
    for k = 1:numel(w_list)
        o = struct('A', A_pair(q), 'w0', w_list(k), 'min_err', 500, 'max_bits', 4e4);
        o.use_pn = false; W0(q,k) = binary_comm_sim(M, a, o);
        o.use_pn = true; W1(q,k) = binary_comm_sim(M, a, o);
    end
    for k = 1:numel(w_fine)
        [Wt0(q,k), Wt1(q,k)] = ber_theory_jam(M, A_pair(q), w_fine(k), Pn);
    end
end
fprintf('Experiment 3 done\n');

f = figure('visible', 'off');
floor_ = 1e-5;
for q = 1:2
    subplot(2,1,q);
    h1 = semilogy(w_fine/pi, max(Wt0(q,:), floor_), 'r-', 'LineWidth', 0.8); hold on;
    h2 = semilogy(w_list/pi, max(W0(q,:), floor_), 'rx', 'MarkerSize', 5);
    h3 = semilogy(w_fine/pi, max(Wt1(q,:), floor_), 'b-', 'LineWidth', 1.4);
    h4 = semilogy(w_list/pi, max(W1(q,:), floor_), 'bo', 'MarkerSize', 4);
    grid on; ylim([floor_ 1]);
    title(sprintf('M = %d, A = %g, P_n = 1', M, A_pair(q)));
    ylabel('BER');
    if q == 1
        legend([h1 h2 h3 h4], 'No PN (theory)', 'No PN (sim)', 'PN (theory)', 'PN (sim)', ...
               'Location', 'southwest');
    end
end
xlabel('\omega_0 / \pi');
save_fig(f, 'p128_ber_vs_w0');

%% 4. Spectra before and after despreading
M = 100; A = 10; w0 = 0.2*pi; Nb = 400; L = Nb*M; Nfft = 1024;
b = bit_generator(Nb);
s = modulator(b, M);
p = pn_generator(L);
jam = sine_generator(L, A, w0);
w = noise_generator(L, a);
r = s.*p + jam + w; % received (spread) signal
z = r .* p; % after despreading
[Psp, om] = psd_avg(s.*p, Nfft); % transmitted spread signal
Pj = psd_avg(jam, Nfft); % jammer in the channel
Pw = psd_avg(w, Nfft); % noise in the channel
Ps = psd_avg(s, Nfft); % desired signal after despreading
Pjd = psd_avg(jam.*p, Nfft); % jammer after despreading
Pwd = psd_avg(w.*p, Nfft); % noise after despreading
Pz = psd_avg(z, Nfft); % total after despreading
```

```
dB = @(P) 10*log10(P + 1e-6);

f = figure('visible', 'off');
subplot(2,1,1);
h1 = plot(om/pi, dB(Psp), 'b'); hold on;
h2 = plot(om/pi, dB(Pj), 'r');
h3 = plot(om/pi, dB(Pw), 'Color', [0.5 0.5 0.5]);
grid on; ylim([-40 60]); ylabel('PSD (dB)');
title('In the channel (before despreading)');
legend([h1 h2 h3], 's(n)p(n)', 'i(n)', 'w(n)', 'Location', 'northeast');
subplot(2,1,2);
h1 = plot(om/pi, dB(Ps), 'b'); hold on;
h2 = plot(om/pi, dB(Pjd), 'r');
h3 = plot(om/pi, dB(Pwd), 'Color', [0.5 0.5 0.5]);
grid on; ylim([-40 60]); ylabel('PSD (dB)'); xlabel('\omega / \pi');
title('After despreading (input to the demodulator)');
legend([h1 h2 h3], 's(n)', 'i(n)p(n)', 'w(n)p(n)', 'Location', 'northeast');
save_fig(f, 'p128_spectra');

%% Save numeric results
res128 = struct('M_list', M_list, 'w_in', w_in, 'w_out', w_out, 'Pn', Pn, ...
    'T', T, 'Tth', Tth, 'A_list', A_list, 'jsr_db', jsr_db, 'B0', B0, 'B1', B1, ...
    'Th0', Th0, 'Th1', Th1, 'w_list', w_list, 'A_pair', A_pair, ...
    'W0', W0, 'W1', W1, 'Wt0', Wt0, 'w_fine', w_fine, 'Wt1', Wt1, ...
    'psd_om', om, 'Psp', Psp, 'Pj', Pj, 'Pw', Pw, 'Ps', Ps, 'Pjd', Pjd, 'Pwd', Pwd, ...
    'psd_M', M, 'psd_A', A, 'psd_w0', w0);
save(fullfile(fileparts(mfilename('fullpath')), '..', 'results', 'res128.mat'), 'res128', '-v7');
write_table_128(res128);
```

Listing 12: project12_8.m